

CSE 450 CAPSTONE PROJECT · DEPARTMENT OF CSE, BUET

Exploratory Testing Automation



An autonomous agent that decides what to test, runs it on a real Android device, and learns from every run.

TEAM

Mohammad Raihan Rashid · 2105046
Mehemud Azad · 2105014
Jonayed Mohiuddin Shanto · 2105060
Khalid Hasan Tuhin · 2105002
Niloy Kumar Mondal · 2105044

SUPERVISOR

Dr. A. B. M. Alim Al Islam
Professor, Department of CSE, BUET

INDUSTRY PARTNER

Samsung R&D Institute Bangladesh

THE PROBLEM

Why exploratory testing is hard to automate

Manual exploratory testing

Finds real bugs, because a person knows what correct looks like.

But it does not scale, and the knowledge leaves with the tester.

Scripted automation

Repeatable, and cheap to run again.

But it only checks what its author thought to check. It confirms; it does not discover.

Random (Monkey) testing

Explores at scale, with no setup.

But it has no idea what correct looks like. It can report a crash, not a wrong result.

“An automated tool can tell you an application crashed. It cannot tell you it quietly charged the wrong price, because it has no idea what the right price idea what the right price was.”

RELATED WORK

What existing research agents miss

Capability	Research agents today	Our system
Reads the requirements specification	No, so no oracle beyond a crash	Yes: every requirement becomes a queryable node
Remembers across runs	Only within one session	A persistent knowledge graph
Separates its own failures from app defects	No	Three-way fault attribution
Chooses its own test objectives	5 of 7 are handed a task	Yes, and each test cites a requirement

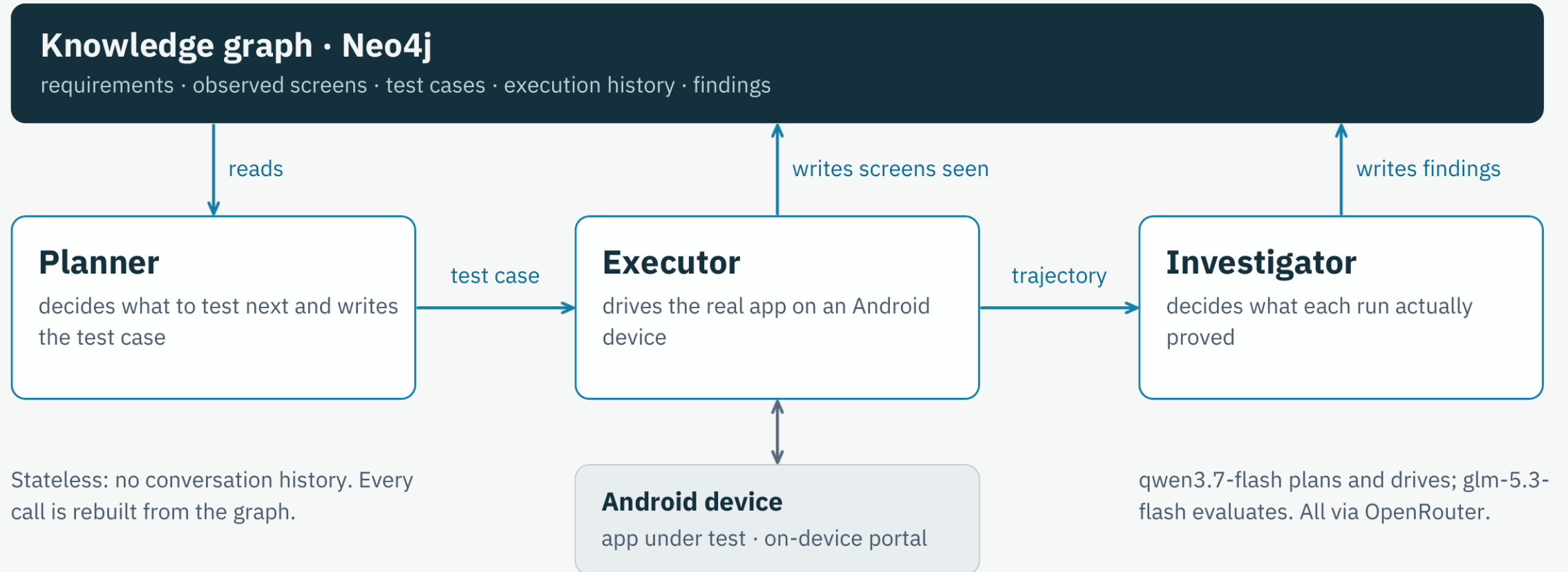
From our review of seven LLM-based GUI testing systems, among them GPTDroid, DroidAgent and AutoDroid.

THE PARTNER'S BRIEF · SAMSUNG R&D INSTITUTE BANGLADESH

**“Every execution cycle must make
make the agent smarter than the
the last.”**

Our answer: three stateless AI agents that never talk to each other. Everything they learn goes into one shared knowledge graph, and every new test is planned from it.

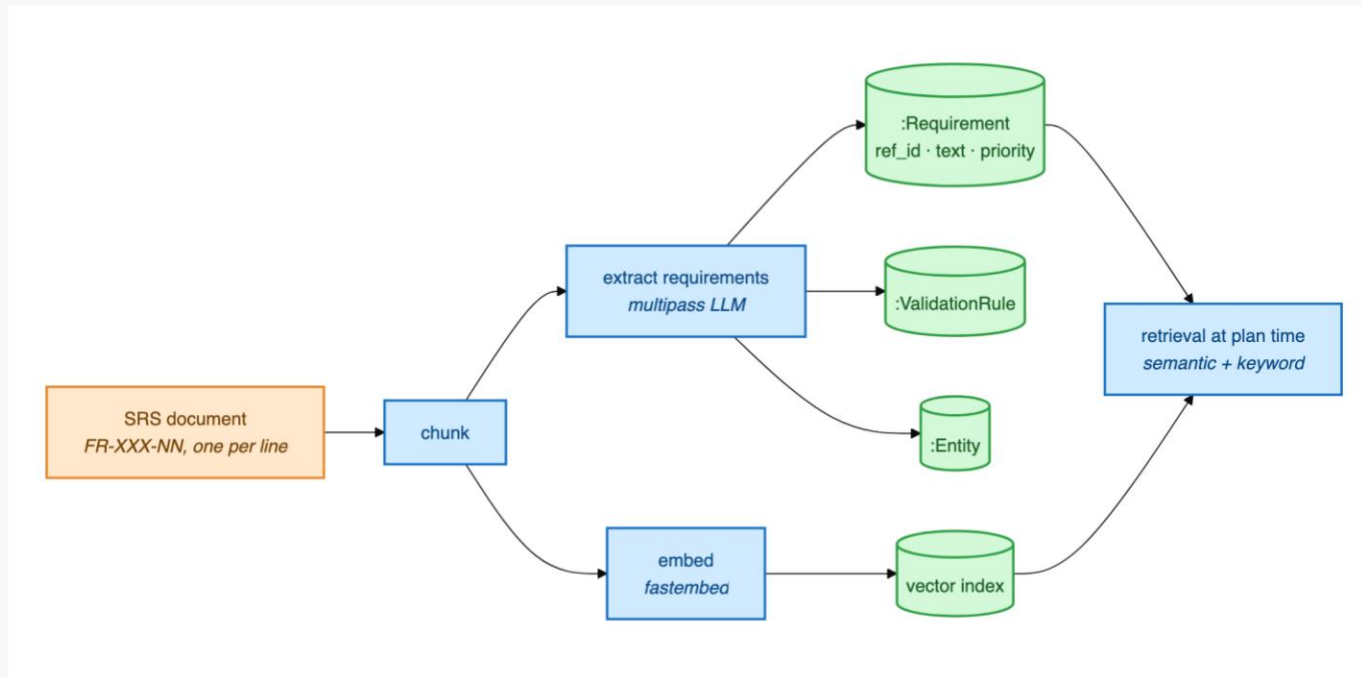
Three agents, one shared memory



INGESTION

A document becomes queryable requirements

Prose in. Individually identified, traceable requirements out.



Each requirement carries

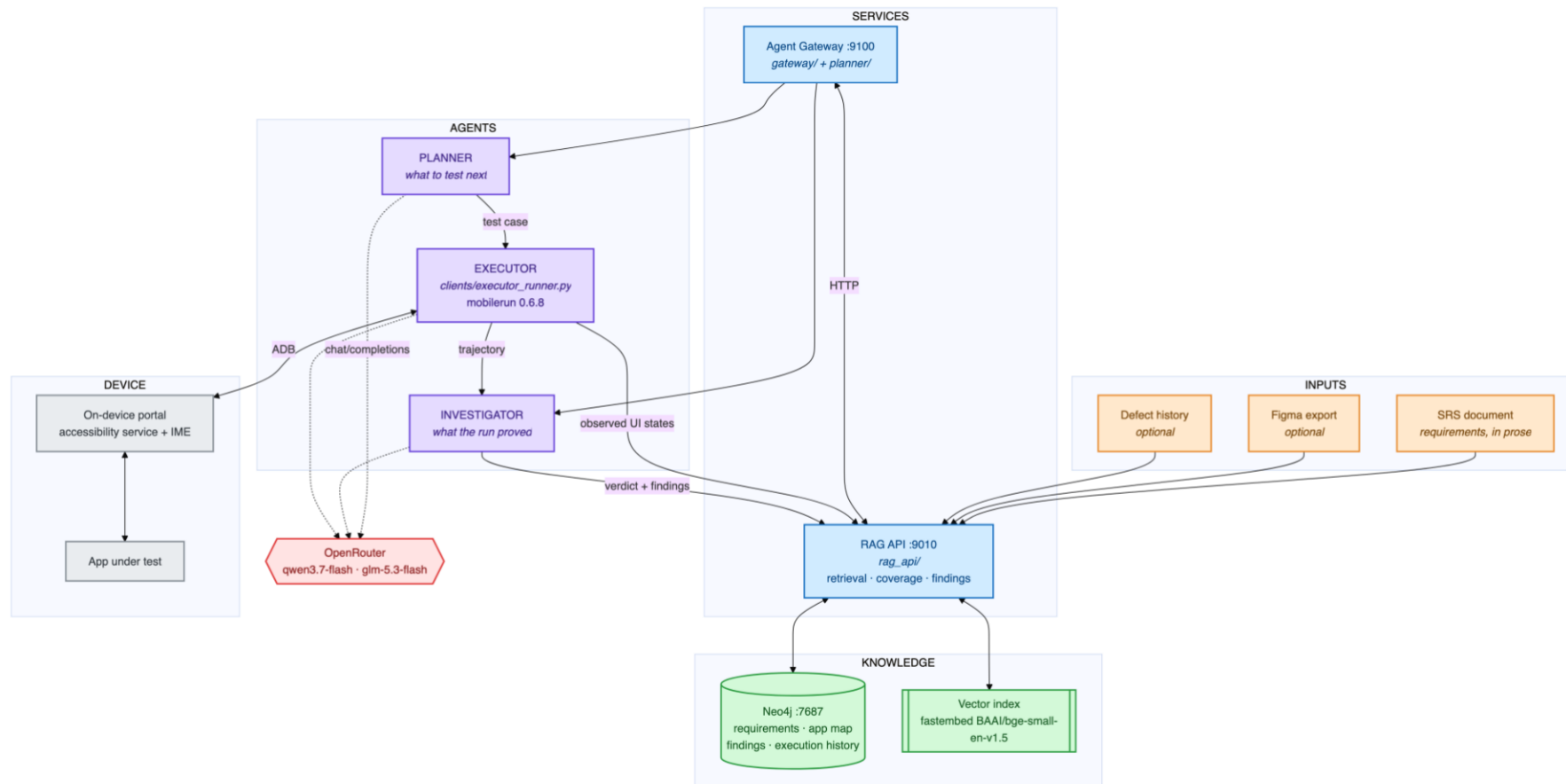
id · type · priority
actor · action
constraints
acceptance
feature area

plus validation rules and the domain entities
they act on

*All three documents are optional. A target with
none of them still runs.*

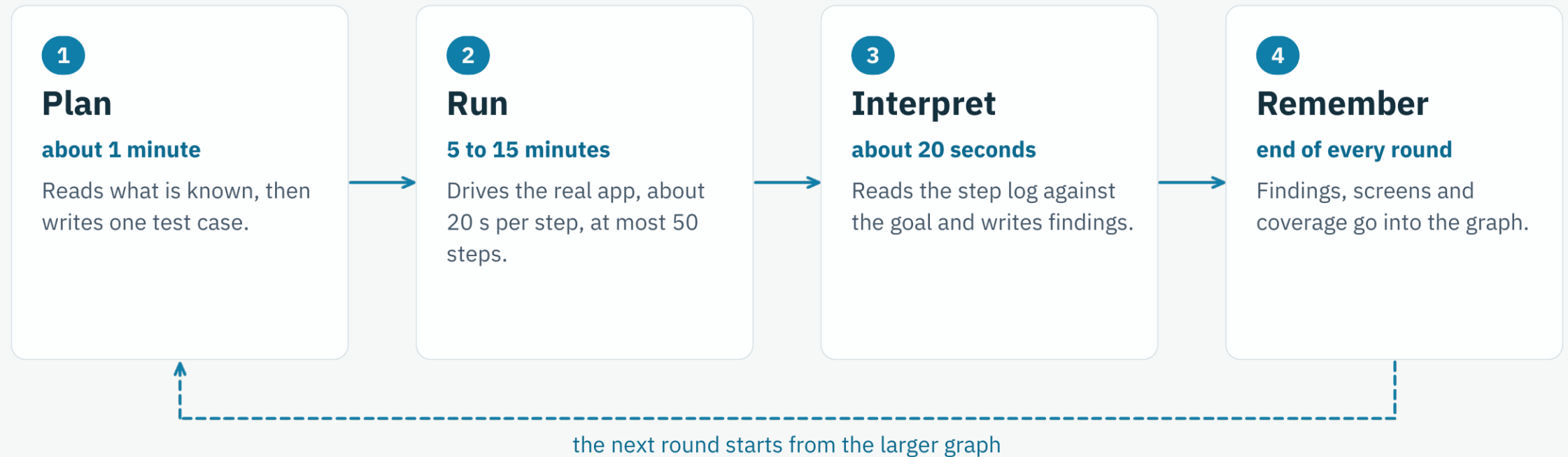
Each requirement links back to the exact chunk of text it came from, which is what makes requirement coverage reportable.

Every component, and how they connect



HOW A TEST RUNS

One round: plan, run, interpret, remember



Every model call is stateless. The agent improves only because the graph grew.

The fourteen messages of one round

1–5 · Planner

proposes and stores the test

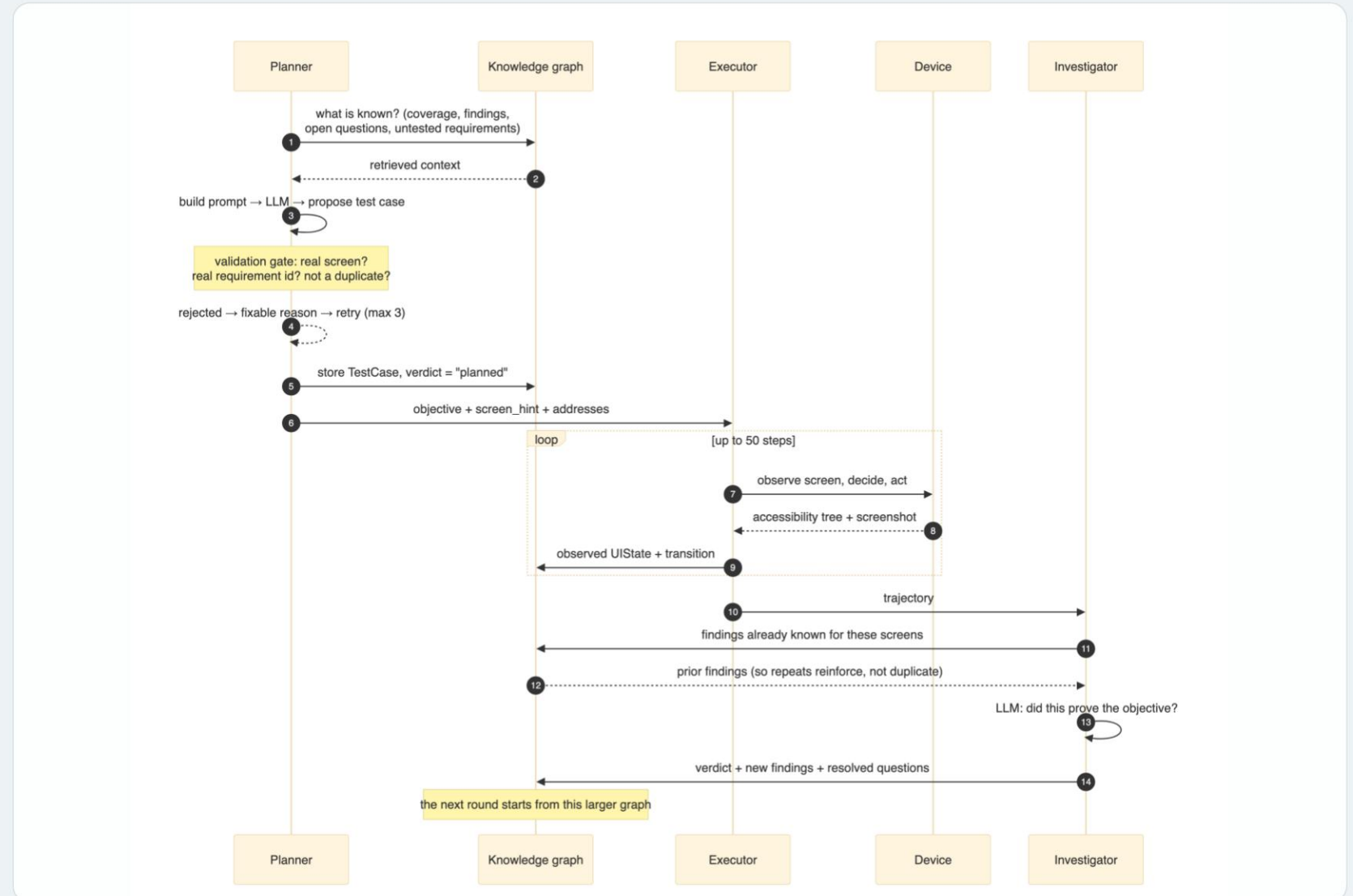
6–9 · Executor

runs it, posting every screen

10–14 · Investigator

judges it, writes findings

Only the graph carries over.



MODELS

Which model does what, and why

Role	Model	Why this choice
Planner	qwen/qwen3.7-flash	Long context and dependable structured output matter more than deep reasoning. About 8% of a test's cost.
Executor (Android)	qwen/qwen3.7-flash, with vision	Reads the accessibility tree and a screenshot on every step, so it is about 92% of a test's cost.
Investigator and rate-limit fallback	z-ai/glm-5.3-flash	A different model family, so no model grades its own execution. Low reasoning effort cut an evaluation from 113.8 s to 22.7 s.
Specification extraction	qwen/qwen3-235b-a22b	Run once per document, and every later decision rests on it, so accuracy is worth paying for here.
Embeddings	BAAI/bge-small-en-v1.5	Runs locally through fastembed, with no per-call charge.

All routed through OpenRouter. Every campaign in this deck ran on the planner, executor and investigator above; the planner is next moving to deepseek-v4-flash.

What the planner knows before it writes a test

Specification

Every requirement as a node, retrieved by meaning and by keyword.

Screens seen

The live map of the app: 79 screens and 797 transitions on the contacts app.

Navigation paths

Learned routes to each screen, and the routes known to fail.

Findings

What earlier runs confirmed, contradicted or left open.

Coverage

Which requirements and feature areas are still untested.

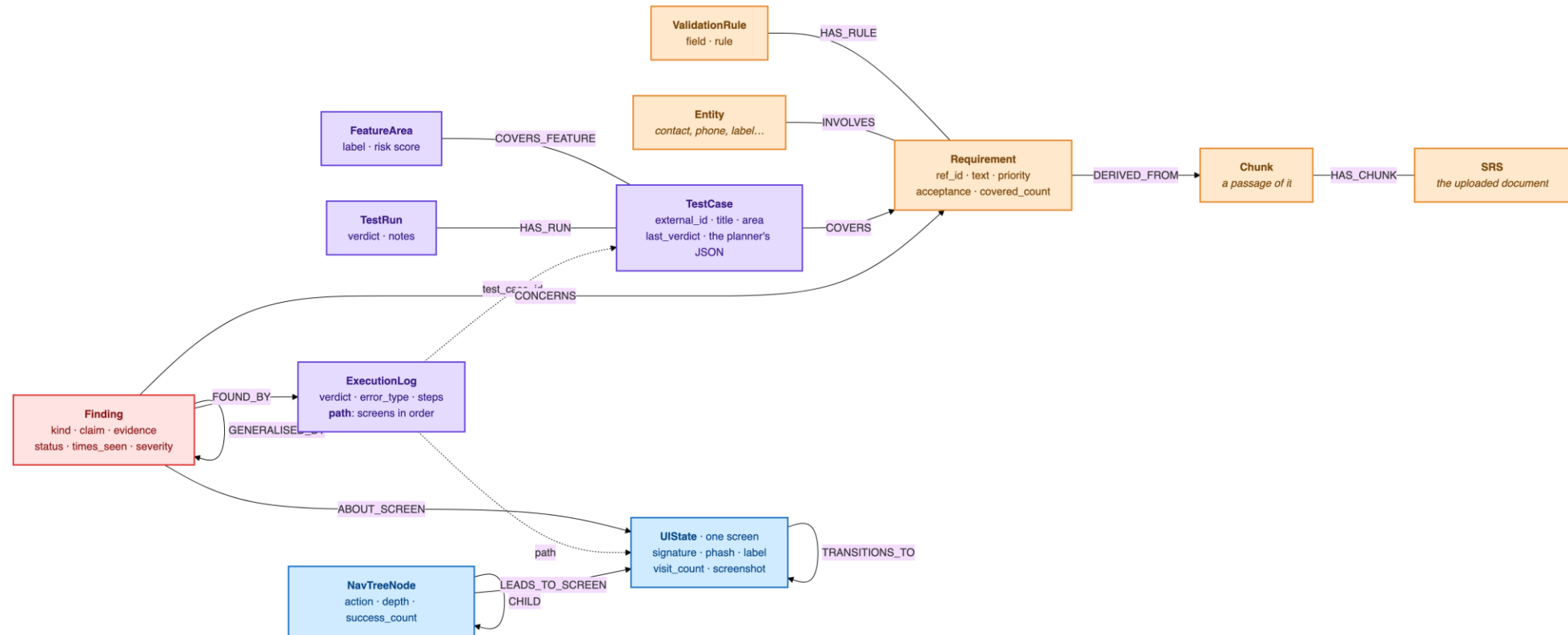
Defect history

Past bug reports, when a project has them.

Bounded context, so long campaigns stay cheap

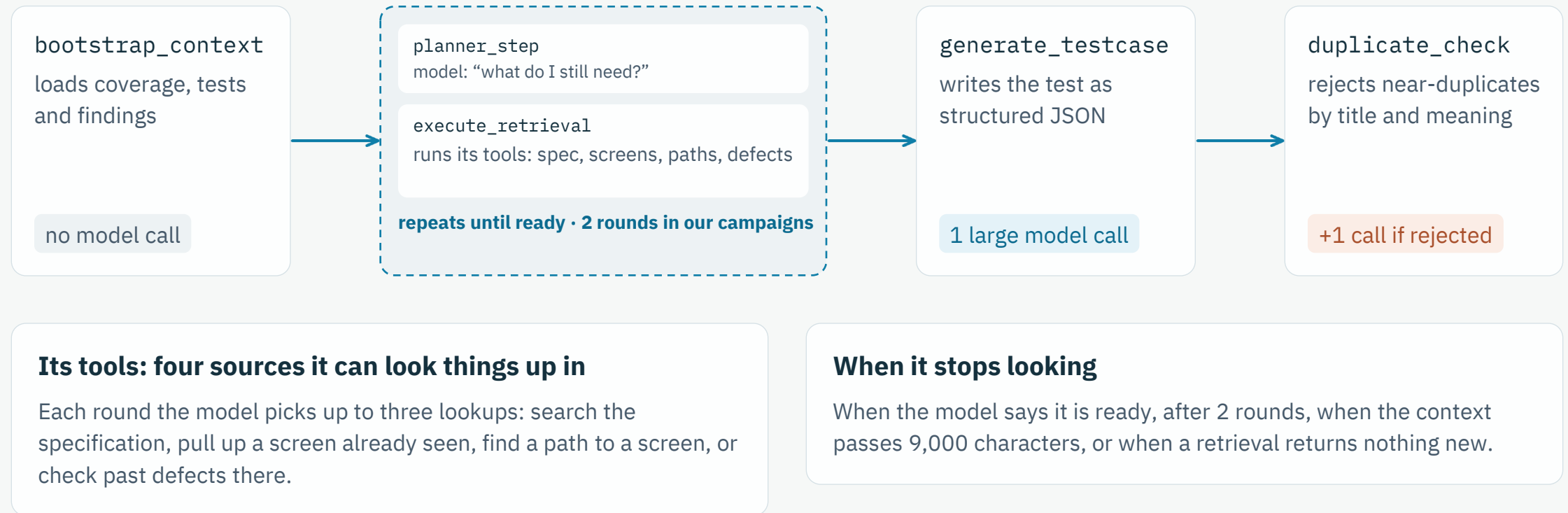
The block summarising earlier runs was 35,233 characters, 69% of one prompt. Stored as findings instead of prose, it is now about 2,000.

What the knowledge graph stores

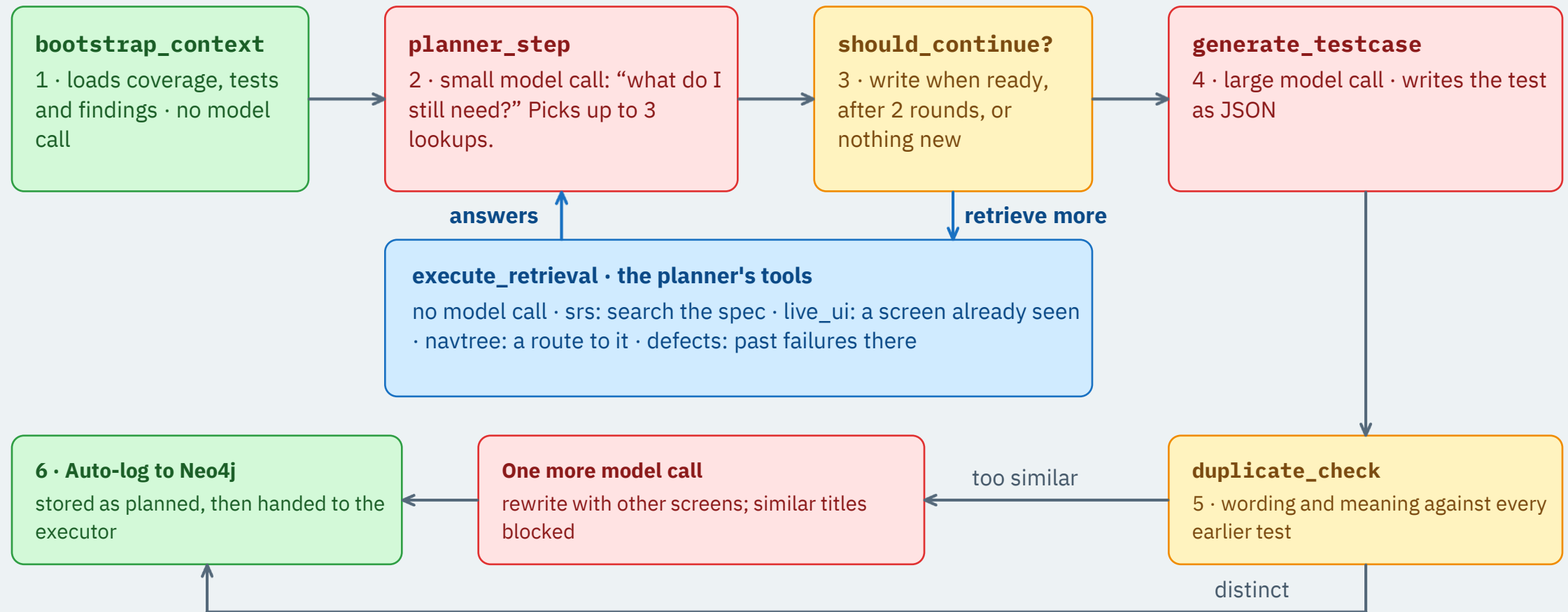


Orange: from the specification · Purple: tests and runs · Blue: the app model · Red: findings · Dashed: stored as a property, not a relationship. Every node belongs to one project.

The planner asks what it still needs



The full planning pipeline



Tools philosophy: the model asks the graph itself

It calls one tool, reads the answer, then chooses the next question. Nine read-only tools, each offered only when its source has data.

Tool	What it answers	Offered when
search_requirements	semantic and keyword search over the specification	a spec is ingested
list_untested_requirements	the coverage frontier: what no executed test covers yet	a spec is ingested
get_screen	the real controls observed on one screen	screens have been seen
list_screens	every screen actually seen, with visit counts	screens have been seen
get_nav_path	a route to a screen the executor has already walked	a route exists
findings_summary	counts of everything earlier runs found, per screen and kind	always
list_findings	the claims themselves: confirmed, suspected, unverified	always
list_open_questions	questions earlier runs raised but never settled	always
get_coverage	tests run, failing hot spots, dead ends, exhausted areas	always

One round: investigate, then propose

Given

Starts from a short brief

coverage map, last few runs, open questions

Calls 1–3

Orient: what is known, and where the gaps are

`findings_summary`, `get_coverage`, `list_screens`

Call 4

Pick up an unfinished thread

`list_open_questions`, `list_findings`

Call 5 on

Ground it in a real requirement and screen

`search_requirements`, `get_screen`

Turn 7 on

Forced to commit

the tool choice is pinned to `propose_test_case`

Every proposal is validated

Rejected if it:

- names a screen never observed
- cites a requirement id that does not exist
- repeats an earlier test
- needs a precondition the tester cannot create
- falls outside the agreed scope

The model is told why and retries, up to 3 times.

Measured over 62 rounds on the commerce app: the most common tool at each position, a median of 13 calls per round, 989 calls for 101 proposals. Left free, the first live run investigated for ten turns and never proposed.

Where to test next, without repeating itself

Steered to the gaps

A coverage map and an exploration directive point it at untested requirements and feature areas.

A cap on every area

After 5 tests an area is marked spent and is no longer promoted, whatever its results.

No near-duplicates

Each new title is compared with every earlier test, by word overlap (0.60) and by meaning (0.90 cosine).

Why the cap was needed

In one 13-round campaign, 9 tests landed on the same screen: a failing area kept being promoted, and nothing ever retired it. The intuitive fix, stopping once an area yields nothing new, would never have fired, because the ninth test was still producing technically new findings.

What the planner actually emits

A test case in plain language, never a tap sequence. Both planner modes emit the same fields.

```
title
objective
expected_result
preconditions[]
screen_hint
area · priority · test_type
requirement_ids[]
rationale
addresses
```

TC-003 · commerce app · tools mode

objective

“Verify that clearing the farm name field to empty and attempting to save save produces a visible error identifying the farm name as required.”

requirement_ids **addresses**

FR-FARM-04 F-422162fd

Requirement ids make requirement coverage reportable. **addresses** names the open question this test settles: an earlier run suspected farm names were never validated.

One real test, from plan to finding

TC-018 · written by the planner

Verify duplicate detection triggers for a phone number match when the display name is intentionally different

Field	What the planner wrote (condensed)
Requirements	FR-DUP-02, FR-DUP-04
Why this test	Earlier runs saw silent duplicates but left phone-only detection unverified. A different name isolates FR-DUP-02 from the name rule.
Objective	Create a contact with a new name but an existing contact's phone number.
Expected	A duplicate dialog shows both records (FR-DUP-04), instead of a silent save.

EXECUTOR

48 steps. Created and saved the contact, then found both entries in the list.

INVESTIGATOR

Objective refuted · spec violation, high severity

ITS VERDICT

"This directly answers the earlier open question: duplicate detection is not triggered by phone-number match alone."



We chose the executor by trying several

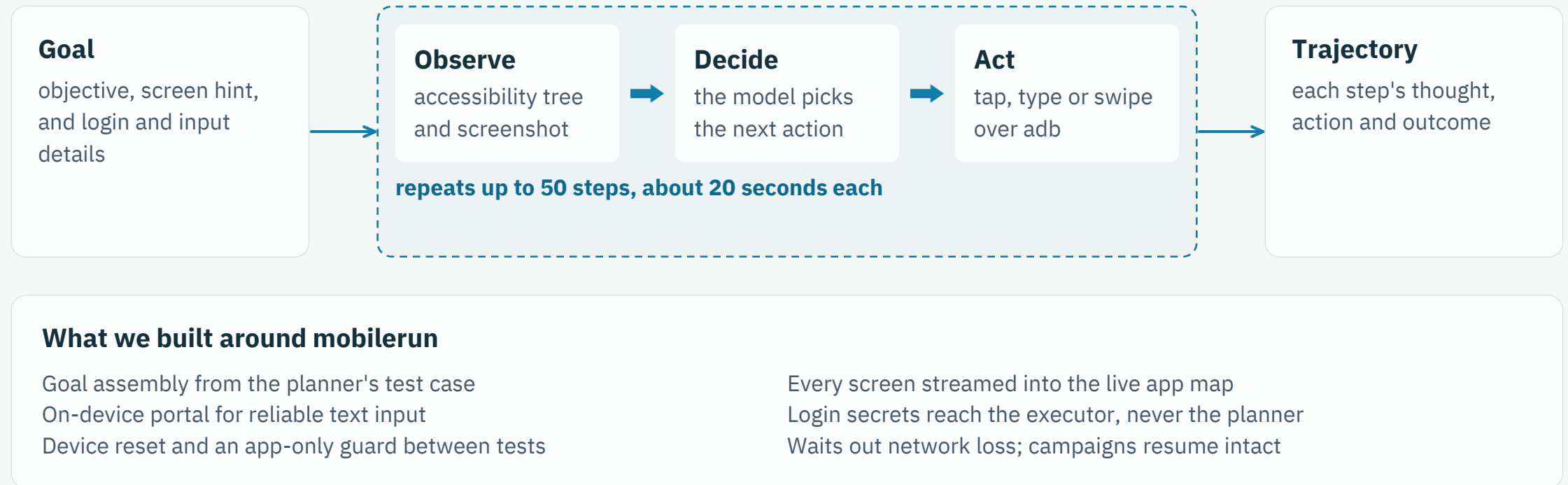
Surveyed benchmarks, then tried hands-on

MUI	evaluated
GeLab	evaluated
MobileAgent-V3	evaluated
mobilerun (Droidrun)	✓ selected

What we needed from it

- Runs from a goal in plain language, not a script
- Drives a real Android device through its accessibility tree and screenshots
- Accepts our own model provider (OpenRouter)
- Saves a step-by-step trajectory we can analyse
- Streams every screen it sees, so we can map the app

How a test is executed on the device



What the executor is actually given

TC-002, contacts app. Assembled from the planner's test case and the target profile.

```
Open the Contacts app on this device.  
Navigate to the 'Search Results' screen if not already there.  
  
Preconditions: App is open.  
Step 1: Navigate to 'Contacts List' screen.  
Step 2: Tap 'FAB: Create Contact' to create a new contact.  
Step 3: Enter 'First name' as 'Alice'.  
Step 4: Tap 'Save' button.  
Step 5: Navigate back to 'Contacts List'.  
Step 6: Tap the search input field to focus.  
Step 7: Type 'Bob' into the search input.  
Step 8: Observe the search results area.  
  
Expected result: Search results should be empty. The app should  
show an empty state and not crash or show any error.
```

Thought, action, outcome, every step

```
Step 1/30  "I'm on a contact detail page for 'john doe'
           instead of the Contacts List."
           click(index=21) -> 'Navigate up' | Button | (74, 226)

Step 2/30  "I can see the FAB 'Create contact' at index 76."
           click(index=76) -> 'Create contact' | FAB | (943, 2076)

Step 3/30  "'First name' EditText is at index 30."
           click(index=30) -> focused, then typed 'Alice'
```

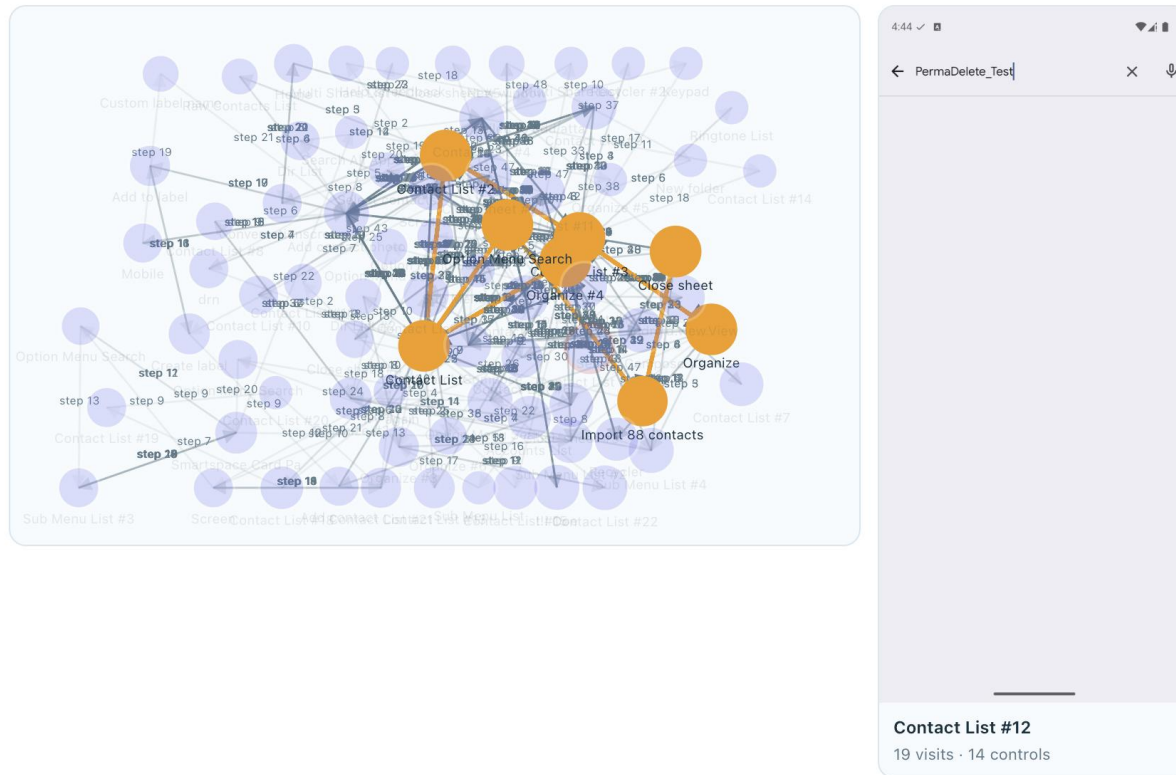
Result: the search area is empty, no contacts matching "Bob". The app did not crash and no error occurred.

Every run's path through the map

HOW IT EXPLORED

Live App Model (79 states · 797 transitions)

The self-built map of screens the agent has actually reached (deduped by structural signature — scrolling & theme don't create duplicates). Drag nodes; click a state to see its screenshot. Highlighting the path of TC-088.



79 screens · 797 transitions

The contacts app, mapped only from screens the agent actually reached.

Pick a run, see its path

In orange, the screens TC-088 visited, in order, read from that run's log.

Click a screen

Its stored screenshot, how often it was visited, and its controls.

Is this a screen we have seen?

1	Exactly the same? every control matches, in the same order	exact	match · same screen
2	Almost the same? ignores counters, timestamps and list data that change on their own	90% shared	match · same screen
3	Is one part of the other? catches a screen captured before it finished drawing	95% contained	match · same screen
4	Same controls, different values? compares what the controls are, not the text typed into them	95% by type	match · same screen
5	Does it simply look the same? an image fingerprint, for screens that expose no usable labels	near identical	match · same screen

No match on any of the five: a screen we have not seen before. Store it, and record how we got here.



The same planner, a browser instead of a device

A second executor behind the same contract. The planner does not know which one will run.

What it observes

- An accessibility style description of the DOM, not screenshots
- role and name, the same vocabulary the Android executor uses
- native HTML5 validation, including the browser's own message
- a password is described, never echoed

What it can do

- click fill press select
- goto back scroll wait finish
- nine actions, each checked against the target's guardrails
- CAPTCHA escalates to a person when visible, fails cleanly when headless

How it knows something broke

- console errors
- uncaught page errors
- HTTP 4xx and 5xx responses
- dialogs the page never showed

Those signals are collected independently of anything the agent claims, so a 5xx can override a claimed pass. On the live site this caught a real HTTP 500.

Testing Bangla text on a live survey platform

TC-001 on DataGhurhi. The same thought, action, outcome loop, over the DOM.

```
[1/80] dataghurhi.cse.buet.ac.bd
  think: find a survey with a text question to test
         Bangla Unicode input
  act:   {"action":"click","ref":"e9"}
  ->    clicked [e9] button "Test Project for Survey"

[4/80] /view-survey/601
  act:   {"action":"scroll","direction":"down"}
  ->    scrolled down           page: 56 controls

[5/80] /view-survey/601
  act:   {"action":"select","ref":"e20","value":"Text"}
  ->    selected 'Text' in the question type dropdown

[7/80] /view-survey/601
  act:   {"action":"fill","ref":"e19","text":"আপনার নাম লিখুন"}
  ->    filled "Enter your question here"
```

Controls by reference, not pixels

Each observed control gets a stable reference the model addresses by name.

Non Latin text end to end

Bangla is typed, stored and read back, then checked character for character.

Bounded like the device runs

Up to 80 steps, with guards that end a run repeating a dead action or wandering.

5 targets · 2 platforms · 1 planner

INVESTIGATOR

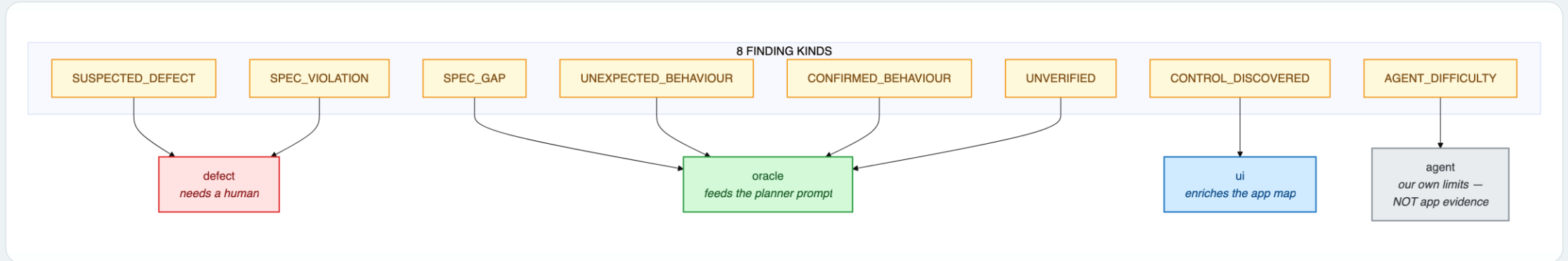
The investigator turns runs into knowledge

- 1 Reads the run**
Compares the device's step log with what the test was meant to verify.
- 2 Writes typed findings**
Each is one atomic claim. A repeat observation reinforces an existing finding instead of creating a new one.
- 3 Settles open questions**
A question one run raises is answered by a later run, or retired as inconclusive after three attempts.

Group	Finding kinds	Used for
Defect	SUSPECTED_DEFECT, SPEC_VIOLATION	A human reviews it
Oracle	CONFIRMED_BEHAVIOUR, UNEXPECTED_BEHAVIOUR, SPEC_GAP, UNVERIFIED	Fed back to the planner
UI	CONTROL_DISCOVERED	Adds to the app map
Agent	AGENT_DIFFICULTY	Our own limits, never app evidence

Contacts app so far: 345 findings, including 36 suspected defects and 21 specification violations.

Eight kinds of finding, four destinations



Red: defect

Suspected defects and spec violations go to a human.

Green: oracle

What the app does, or leaves unclear, feeds the planner.

Blue: UI

A newly found control enriches the app map.

Grey: agent

Our own difficulties, never counted as app evidence.

Knowing a finding is the same finding

1 · Shown what is known

The model sees up to 12 known findings for the screens this run touched.

2 · Says what it confirms

A confirmation reinforces the existing finding instead of adding a new one.

3 · Embedding safety net

A new claim at 0.90 cosine or more to one in the same group is merged anyway.

Why the model decides, not similarity alone: a true restatement of one finding scored 0.846, while a genuinely different defect scored 0.853.

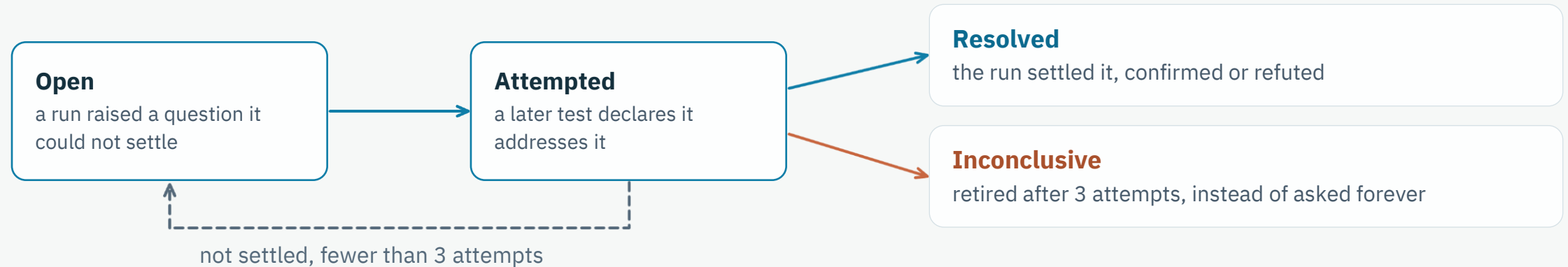
Clusters become one claim

Three or more similar findings on one screen (0.72) can be merged into a general claim. On the contacts app, 5 general claims stand for 26 specific ones.

Findings, not prose, keep input small

With prose reports the investigator's input grew from 28,344 to 97,796 characters over a campaign. With findings it shrank from 20,466 to 10,959.

From observations to conclusions



Open questions become the planner's missions

The planner reserves room for open questions, so older ones are not crowded out by new ones. Its test declares which question it addresses, and the investigator records the answer.

8

settled by a later run

115

still open

Contacts app. The loop works; most questions need longer campaigns.

FAULT ATTRIBUTION

Not every failure is the app's fault

App fault

The agent reached its checkpoint and the app misbehaved. These are the findings.

ASSERTION_FAILURE · CRASH ·
APP_UNRESPONSIVE

11 **32**

commerce

contacts

Agent fault

Our executor could not drive the app. Says nothing about its quality.

NAVIGATION_FAILURE ·
ELEMENT_NOT_FOUND · TIMEOUT ·
NAVIGATION_LIVELock

2 **6**

commerce

contacts

Environment

A precondition was missing, or the step budget ran out before any evidence.

PRECONDITION_NOT_MET ·
PERMISSION_DENIED ·
STEP_LIMIT_EXCEEDED

12 **10**

commerce

contacts

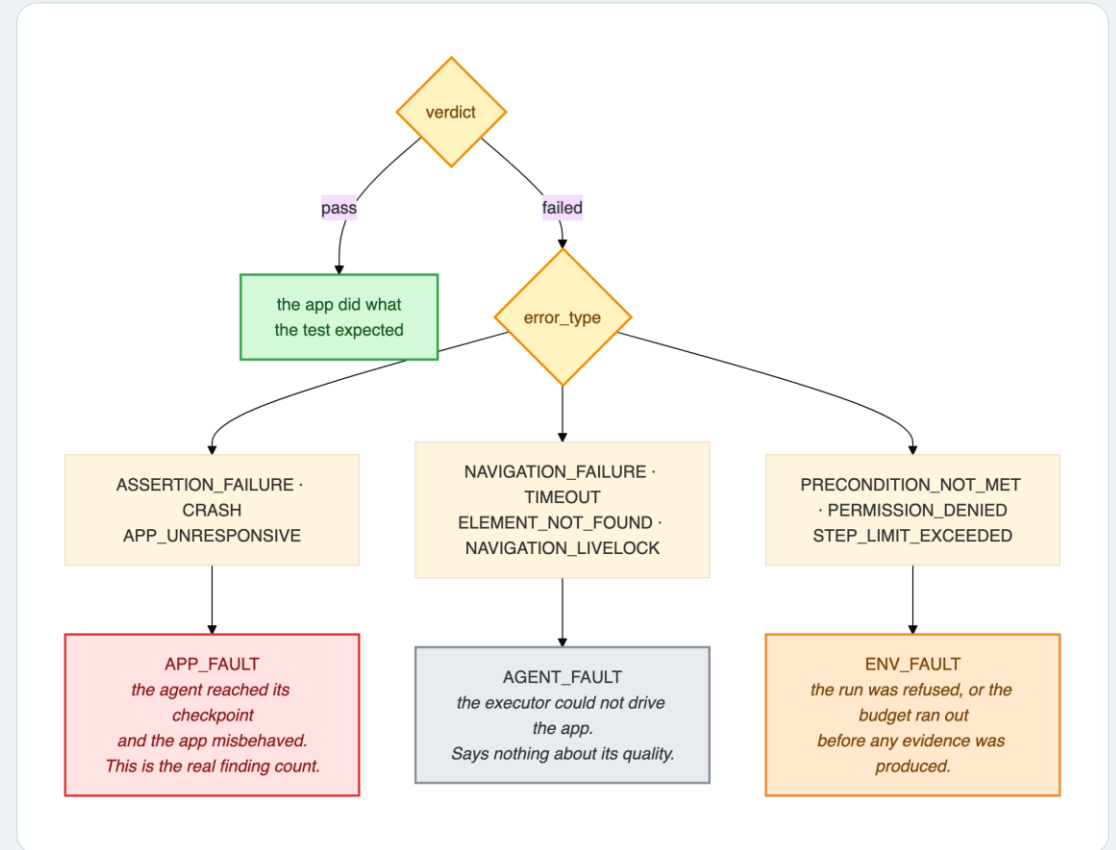
Only the first column counts as defects: 11 of 25 failures on the commerce app, 32 of 48 on the contacts app.

Every error type, and where it goes

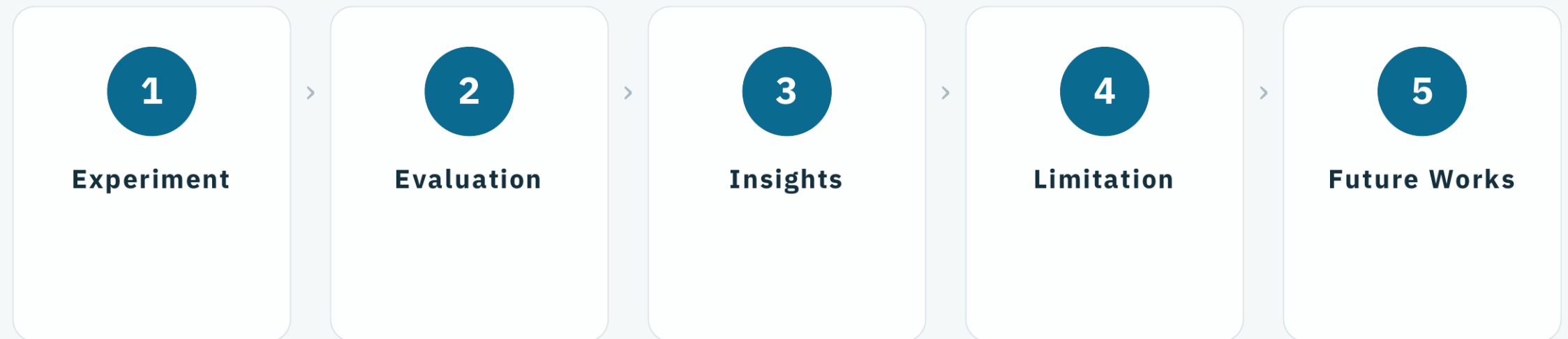
How to read it

- Green: pass, the app did what the test expected
- Red: app fault, the real finding count
- Grey: agent fault, our executor could not drive the app
- Orange: environment fault, refused or out of budget before any evidence

The taxonomy has a single definition in the code, so the dashboard, the reports and these slides all count the same way.



Contents



189 autonomous runs across five targets

Target	Platform	Tests	Passed	App fault	Agent fault
DataGhurhi, survey platform (Deployed)	Web	64	51	4	0
Shobar Khamar, marketplace (Deployed)	Android	45	20	11	14
Google Contacts (Deployed)	Android	50	20	19	10
broken-workshop, (no specification, srs, figma)	Web	10	5	0	4
AcademyBug, (no specification, srs, figma)	Web	20	1	3	16

The two end rows are the whole argument

With a specification

0

agent faults in 64 runs.

Without one

16

agent faults in 20 runs.

The specification is not decoration. It is what lets the agent know where it is going.

USD 0.008

per executed test, measured against the provider's billing record at 10.1 device steps per test.

Comparable published work reports about USD 18 per application, and states that the cost prevented the authors from repeating their own runs.

Limitations and next steps

What we cannot claim yet

- No ground truth: both apps are closed-source, so precision and recall cannot be computed.
- No ablation yet: we have not measured what each component contributes.

What comes next

- An open-source benchmark with known, reproducible bugs (Themis) for bug-detection rate.
- Ablations: findings off, memory off, and a random baseline.

Thank you

Questions are welcome.

With thanks to our supervisor, Dr. A. B. M. Alim Al Islam, and to Samsung R&D Institute Bangladesh.